MAX Jeune Journey Planner: Code Audit Report

Audit date: 11 October 2026. Subject: `sncf_max_jeune.zip` (Node.js server, planner engine, single-page frontend, Electron desktop wrapper). No source files were modified; all execution happened in a scratch copy.

1. Executive summary

The project is a working, thoughtfully scoped local tool. Its data handling is honest about uncertainty (the UI repeatedly says MAX availability is a daily snapshot), output escaping in the frontend is consistent, the server binds to loopback only, and the Electron window uses `contextIsolation` and `sandbox`. The routing core is clever and mostly correct on the cases it tests.

The main risks are operational rather than algorithmic: the timetable is downloaded once and never refreshed, a single search can freeze the whole process for a minute or more, the transfer-time logic that production actually uses is untested and does not match what the README promises, and the HTTP layer has no input validation or top-level error handling.

23 findings: 3 High, 13 Medium, 7 Low.

Fix first, in this order:

1. **F-01** Refresh the GTFS timetable (it silently expires on 31 March 2027 and is never updated before then).
2. **F-02** Stop building legs for every day between outbound and return (66 s for a 30-day span, measured), and move the work off the main thread.
3. **F-03** Make the default transfer-time path match the documented rules and cover it with tests.
4. **F-04 to F-07** Add a top-level error handler, validate every query parameter, validate the `Host` header, and return 400 for bad input.
5. **F-19** Fix the one failing test so the suite is green and trustworthy again.

2. Scope and method

Reviewed by reading: `server.js` (81 lines, some over 2,700 characters), `planner.js` (301 lines), `index.html` (165 lines), `desktop/main.js`, `forge.config.cjs`, `package.json`, `README.md`, `test/route-search.test.js`, and the three `audit-*.js` scripts. `node_modules/`, `out/` and the macOS build artefacts were not reviewed.

Executed (in scratch):

- `node --test`: 18 tests, 17 pass, 1 fails (F-19).
- The three project audit scripts against the shipped cache.
- Targeted probes of `planner.js` functions (transfer rules, time handling, station keys).
- The server in its built-in sample mode on a local port, probed with `curl` and raw sockets.
- The real cached GTFS feed (525,244 `stop_times` rows) run through `makeJourneyLegs` to measure cost and correctness.
- `npm audit` against `package-lock.json`.

Not done: live calls to SNCF endpoints (network blocked in the audit environment), running the Electron GUI, and auditing the bundled third-party `sncf-max-main.zip`.

Each finding is tagged **Verified** (I executed it and saw the behaviour) or **By inspection** (read from the code, not executed). Line numbers refer to the files as delivered.

Severity: High = affects most users or most searches, or loses/blocks function. Medium = real defect or risk with a limited trigger or limited impact. Low = polish, latent bug, or maintainability.

3. Findings at a glance

ID	Severity	Area	Finding
F-01	High	Data	GTFS timetable is never refreshed after first download
F-02	High	Performance	Search cost scales with the whole date span and blocks the event loop
F-03	High	Correctness	Production transfer logic is untested and diverges from the documented 90-minute Paris rule
F-04	Medium	Reliability	Unhandled errors in the request handler can kill the server
F-05	Medium	Security	Local API accepts any Host, any method, no headers
F-06	Medium	Input validation	`/api/search` dates unvalidated, interpolated into a query, no span limit
F-07	Medium	API design	Bad user input returns HTTP 502

ID	Severity	Area	Finding
F-08	Medium	Data	MAX cache is single-slot; a cache write failure discards a good download
F-09	Medium	Reliability	Concurrent and non-atomic timetable downloads
F-10	Medium	Security	Electron navigation guard can be bypassed
F-11	Medium	Correctness	Daylight-saving and machine-timezone handling
F-12	Medium	Correctness	GTFS boarding and alighting restrictions are ignored
F-13	Medium	Correctness	Route search truncates silently and can wander for days
F-14	Low	Correctness	`HH:MM` GTFS times are mis-parsed (latent)
F-15	Low	Correctness	Station-name normalisation mangles some names; hand-kept alias tables
F-16	Low	Correctness	Coaches shown as rail; train number depends on `trip_headsign`
F-17	Low	Reliability	Connection-rules cache can be built from empty data and retries on every call
F-18	Low	Frontend	Stale-response race, no station validation, accessibility gaps, small UI bugs
F-19	Medium	Tests	One failing test; server and rules engine have no coverage
F-20	Low	Docs	README statements that the code or data do not support
F-21	Medium	Maintainability	Dense one-line style and copy-pasted logic
F-22	Medium	Dependencies	31 advisories in build-time dependencies
F-23	Low	Build and hygiene	Unsigned build, external `unzip`, no CI or engines, stray artefacts

4. Detailed findings

4.1 High severity

F-01 GTFS timetable is never refreshed (`server.js:21` , By inspection plus data check)

`loadTimetable()` only calls `fetchGtfs()` when `.cache/gtfs/stops.txt` does not exist. After the first download the 6-hour in-memory cache just re-reads the same files from disk. The shipped feed declares `feed_start_date 20261006` and `feed_end_date 20270331`.

Impact: timetable changes, engineering works and added services are never picked up, and after 31 March 2027 searches return nothing. In the packaged Electron app the cache lives in `userData`, so there is no natural reset. The connection-rules dataset, by contrast, does refresh every 24 hours, so the two data sources are treated inconsistently.

Recommendation: store a fetch timestamp (or compare `feed_info.txt` and HTTP `Last-Modified`), refresh daily, and fall back to the old copy only on network failure with a visible "timetable is N days old" notice.

F-02 Search cost scales with the whole date span and blocks everything (`server.js:65-67`, `planner.js:191-218`, Verified)

`makeJourneyLegs` builds timetable legs for every day from outbound to return+2, although only the outbound and return days matter. The UI lets the return date be up to 28 days after outbound (`index.html:98`). `loadMax` also downloads MAX records for the whole range. I measured the leg builder on the real cached feed (Paris Gare de Lyon to Lyon Part-Dieu):

Date span	Legs built	Build time
3 days	14,816	6.9 s
10 days	39,863	21.3 s
30 days	113,045	66.4 s

This is purely synchronous CPU work on the same thread that serves HTTP. While it runs, no other request (including `/api/health`) is answered. In the Electron app the server runs in the main process, so the app can appear frozen (macOS beachball). Leg generation also enumerates every stop pair per trip ($O(n^2)$) before the cheap relevance filter runs (`planner.js:196-210`).

Recommendation: generate legs only for the outbound window and the return window; filter by `relevant` before building `Date` objects; cache per-day legs; run the engine in a `worker_threads` worker or child process.

F-03 Production transfer logic is untested and does not enforce the documented Paris rule (`planner.js:124, 155-158`, Verified)

`minChange()` encodes the README's rules (90 minutes between different Paris stations, 45 for other station changes, 12 within a station). But `connectionMinimum()` only uses it when `connectionSettings` is null. The server always passes a settings object, so in the default "SNCF rules" mode the answer comes from `sncfConnectionRule()`, whose fallback is **5 minutes** whenever no rule matches or a station's UIC code is ambiguous (for example the "Paris (all stations)" group).

Probe result: a Paris Gare de Lyon to Paris Gare du Nord change with a 6-minute gap is rejected by the legacy path (0 journeys) but accepted in default SNCF mode when no rule is found (1 journey). The README states Paris cross-station transfers need at least 90 minutes.

Every test calls `findJourneys` without `connectionSettings`, so the tests exercise only the legacy path. The ~60 lines that decide transfer feasibility in real use (`buildConnectionTimeRules`, `stationUic`, `sncfConnectionRule`) have no tests at all.

Recommendation: decide which behaviour is intended, make the settings path apply a sensible floor (for example `max(rule, minChange)` when no specific rule exists), and add tests that pass realistic `connectionSettings` and rule tables.

4.2 Medium severity: reliability and security

F-04 Unhandled errors can kill the server (`server.js:36, 80`, Verified)

The request handler is `async` with no top-level `try/catch`. `new URL(req.url, http://${req.headers.host})` throws on a malformed `Host` header. I sent `Host: [` and the process exited with `TypeError: Invalid URL`; the next request got no response. The `/` route (`readFile` of `index.html`) is also unguarded, and `app.listen` has no `'error'` listener (a busy port 3000 would crash `npm start`). A browser never sends such a header, so this needs a raw client, but one bad request should never terminate the service. How Electron's main process reacts to the same rejection was not tested.

Recommendation: wrap the handler body, build the URL from a fixed base (`http://127.0.0.1`), handle `server.on('error')`.

F-05 Local API accepts any caller (`server.js:36-81`, Verified)

The `Host` header is not checked (a request with `Host: evil.example.com` is served), any HTTP method is accepted, and responses carry no `Content-Security-Policy`, `X-Content-Type-Options` or `Cache-Control`. Because the server is on a fixed port 3000 under `npm start`, any web page the user visits can fire simple GET requests at it. Responses are not readable cross-origin without DNS rebinding, but the page can trigger the expensive searches in F-02 as a CPU denial of service, and a rebinding page could

read results. The data exposed is public SNCF data, so confidentiality impact is low; resource abuse is the real issue. The Electron build uses a random port, which reduces but does not remove this.

Recommendation: allow only `127.0.0.1:<port>` and `localhost:<port>` in `Host`, restrict to GET, add a CSP and the usual headers.

F-06 `/api/search` parameters are not validated (`server.js:19, 61-65`, Verified in sample mode, live path By inspection)

`/api/discovery` validates dates; `/api/search` does not. Garbage dates (`outbound=garbage`) and a value containing a quote returned HTTP 200 with empty results. In live mode these strings are interpolated into the ODSQL `where` clause (`date >= '${start}'`) sent to SNCF Open Data, so a crafted value changes the query (public data, so low impact) or produces `NaN-NaN-NaN` dates that fail confusingly. There is also no upper bound on the span between outbound and return, which feeds F-02. `changes=a,b` on the discovery endpoints is accepted and echoed as `[null,null]`.

Recommendation: one shared `parseDate()` and `parseChanges()` used by every endpoint; reject spans over the 30-day window.

F-07 Bad input returns HTTP 502 (`server.js:28-31, 43, 78`, Verified)

Validation errors thrown by `connectionSettings()` and `minimumStay()` sit inside the `try` blocks that map every error to 502. A bad manual minimum returned `{"error":"Minimum connection time must be..."}` with status 502 (Bad Gateway). Use 400 for client errors and reserve 502/503 for upstream failures.

F-08 MAX availability cache design (`server.js:19`, By inspection)

- One in-memory slot and one on-disk file, keyed by exact `start` / `end`. Opening the calendar view (a month range) evicts the range a journey search just loaded, and the next search re-downloads (up to 120 s timeout each). The shipped `max.json` covers a single day (`2026-10-08` to `2026-10-08`), which shows how narrow the disk cache is.
- The `catch` wraps both the download and the cache write. A failed `mkdir` / `writeFile` (full disk, read-only folder) discards a successful download and reports "Could not load SNCF MAX availability".
- An empty export is treated as an error, so a date with no data surfaces as a 502 instead of "no data for this date".
- The disk fallback has no age limit and the cache file is written non-atomically.

F-09 Concurrent and non-atomic downloads (`server.js:20-24`, By inspection)

There is no in-flight promise sharing. On first run the page calls `/api/stations` on load (`index.html:106`) and the user may start a search at once; both reach `fetchGtfs()` and write the same `sncf-gtfs.zip` and `gtfs-new` directory. `fetchGtfs` also renames files into place one by one, so a failure part-way leaves a mix of old and new files, and the connection-rules refresh does `rm` then `rename`, so a failed rename leaves no data. Keep a single shared promise per download and swap a whole directory atomically.

F-10 Electron navigation guard bypass (`desktop/main.js`, `openExternalLinksInBrowser`, Verified)

The `will-navigate` handler allows navigation when `url.startsWith(plannerUrl)`. I confirmed that `http://127.0.0.1:5000@evil.example.com/` passes (its real host is `evil.example.com`) and so does any other local port sharing the prefix (`http://127.0.0.1:50001/...`). Exploiting it needs a way to trigger navigation inside the window, which the app does not currently offer, so exposure is low, but the guard does not do what it claims. Compare `new URL(url).origin` with `new URL(plannerUrl).origin` instead. Positives: `contextIsolation: true`, `nodeIntegration: false` and `sandbox: true` are all set correctly.

4.3 Medium severity: correctness

F-11 Daylight saving and machine timezone (`planner.js:100, 106, 111`; `server.js:43, 51`, Verified)

SNCF times are Paris wall-clock times but are parsed as machine-local dates, and overnight arrivals add a fixed `86400000` ms. Under `TZ=Europe/Paris` a 22:00 to 06:00 night train that crosses the 24/25 October 2026 clock change shows 480 minutes instead of the real 540; across the 28/29 March change it shows 480 instead of 420. 25 October 2026 is inside the current 30-day window. Times in the ambiguous 02:00 to 03:00 hour resolve to one of two instants. Separately, the "only trains leaving after now" cutoff compares the real current instant with Paris wall-clock times, so it is wrong whenever the computer's timezone is not Europe/Paris. Parse with an explicit `Europe/Paris` zone (for example Temporal or Luxon) and compute durations in absolute time.

F-12 Boarding and alighting restrictions ignored (`planner.js:195-215`, Verified)

GTFS `stop_times` carries `pickup_type` and `drop_off_type`. In the shipped feed 63,521 of 525,244 rows (12%) have `pickup_type=1` and 63,376 have `drop_off_type=1`. The planner creates a leg for every stop pair on a trip without checking either flag. On real searches 63 of 10,108 (0.6%) and 31 of 9,020 (0.3%) of the non-MAX timetable legs boarded at a no-pickup stop or alighted at a no-drop-off stop, so impossible paid legs can appear in results and connections. Skip stops whose flag is `1` as origin or destination respectively.

F-13 Route search truncates silently and can wander (`planner.js:228-232` ; `server.js:69` , By inspection)

`findJourneys` stops at 60,000 expansions or 1,200 results and keeps only the first 12,000 frontier paths per depth. The frontier order is the input leg order (MAX legs first, then timetable legs by day), not best-first, so good itineraries can be dropped with no signal to the user. For multi-day trips the outbound search window extends to the return date, and with the default "no maximum wait" setting the engine may chain layovers of days before returning to the cap. Sort the frontier by arrival time or duration, apply the caps after ranking, and surface a "results truncated" flag.

4.4 Low severity: correctness and reliability

F-14 `HH:MM` times mis-parsed (`planner.js:111` , Verified, latent)

With a two-part time the default `s='0'` is a string, so `h*3600+m*60+(s||0)` concatenates. A GTFS time of `10:00` produced a departure of 04:00 on a different date. SNCF's feed uses `HH:MM:SS` today, so nothing breaks now, but the function is fragile. Default `s` to numeric `0` .

F-15 Station-name handling (`planner.js:6-64, 82` , Verified)

`stationKey()` strips words such as `ville` , `centre` , `saint jean` , which turns "Saint-Jean-de-Luz" into `de luz` and "La Ville-du-Bois" into `la du bois` . Comparisons stay self-consistent because both sides use the same function, but names can collide or fail to match. The ~100 hand-maintained IATA alias entries (`stationIataAliases`) and the separate `iataStationNames` table must be kept in sync by hand, and any new feed code is silently unmapped. Drive both from one table and let the `audit:stations` script run in CI.

F-16 Coaches shown as rail; train-number source (`planner.js:113-116, 174, 202` , Verified on data)

Route type 3 (buses, 129 of the feed's routes) is accepted by `isRail()` and labelled "TER / regional connection (paid)". The shipped `trips.txt` has no `trip_short_name` column, so the train number comes from `trip_headsign` , and if that is empty the opaque `trip_id` is shown to users as the train number. This works with today's feed but rests on an undocumented assumption. `calendar.txt` is also absent (service days come only from `calendar_dates.txt`), and its read failure is swallowed by `.catch(()=>[])` (`server.js:21`).

F-17 Connection-rules cache (`server.js:22-25, 43` , By inspection)

`/api/discovery/times` passes `{stops: []}` to `loadConnectionTimes` when the timetable fails to load, and the resulting rule maps are cached for 24 hours for every later request. Also, when a refresh fails but old files exist, `fetchedAt` stays old, so every subsequent call retries the download (up to 120 s) instead of backing off.

4.5 Frontend, tests, docs, maintainability, build

F-18 Frontend issues (`index.html` , mixed)

- *Stale responses (By inspection):* Explore resubmits on every filter change (`:105, :160`) and `requestSubmit()` ignores a disabled button, so overlapping requests can resolve out of order and draw the wrong map. Use `AbortController` .
- *No station validation:* free-text input is sent as-is; a typo yields "No matching train itinerary" with no hint. The server already computes `originMatches` / `destinationMatches` in `diagnostics` but the UI ignores them.
- *Map (Verified by calculation, `:129`):* the hard-coded projection puts Munich (lon 11.5) at x≈766, outside the 760-wide viewBox.
- *CSS (`:7`):* `font:700 15px inherit` is invalid (a family cannot be `inherit` inside the shorthand), so browsers drop the declaration.
- *Accessibility:* tabs have `role="tab"` but no tabpanel/ `aria-controls` /arrow-key handling; the `#status` region has no `aria-live` ; map markers are not keyboard-focusable (the list is an alternative).
- *Minor:* `todayIso` is computed once at load, so a page left open past midnight keeps stale date limits; Trainline deep links exist for only two stations (`:109`); six separate `<style>` blocks accumulated by patching.

F-19 Tests: one failing, large blind spots (`test/route-search.test.js:203` , Verified)

`node --test` gives 17 pass, 1 fail: "a day trip requires time at the destination before its return". The default `minimumStayMinutes` is now 180 (`planner.js:258, 278` ; `server.js:31`) but the test still expects a 30-minute stay to be valid. It passes when `minimumStayMinutes: 30` is supplied, so the test is stale rather than the engine wrong. Beyond that: no HTTP-level tests for `server.js` , no tests for the SNCF rules engine (F-03), no tests for DST (F-11) or boarding flags (F-12), no coverage reporting and no CI.

F-20 Documentation drift (`README.md` , Verified)

The README says Paris cross-station changes need 90 minutes (F-03); says the 8 to 31 October snapshot has 314 station codes, while the shipped `max.json` is a single day with 280 codes; says `unzip` must be on `PATH` but the error for a missing binary reads "Could not download SNCF timetables"; and implies the test suite passes (F-19).

F-21 Maintainability (`server.js` , `planner.js` , Verified)

Much of the code is written as long single lines (several over 2,700 characters, for example `server.js:43, 51, 77`), which makes review, diffs and debugging hard. Duplicated logic is a standing source of drift: `validDate` / `nextDay` appear in two handlers, `activeServicesForDate` (server) duplicates `activeServices` (planner), the CSV line parser exists in three files, and `findReturn` is copy-pasted between `findReachableDestinations` and `findReachableDestinationOptions` (`planner.js:274, 293`). `alignSameDay` mutates its inputs. A formatter (Prettier), ESLint and extraction of shared helpers would pay for themselves quickly.

F-22 Dependency advisories (`package-lock.json`, Verified with `npm audit`)

31 advisories: 1 critical (`tar`, hard-link path traversal), 23 high, 4 moderate, 3 low. All are in build-time packages pulled in by `@electron-forge/*` 7.11.2 (`tar`, `tmp`, `braces`, `micromatch`, `fast-glob`, `@electron/packager` and others); the suggested fix is the Forge 8 major version. The runtime has no third-party dependencies and Electron 42.11.12 itself was not flagged, so practical exposure is limited to the machine that builds the app. Plan the Forge upgrade.

F-23 Build and hygiene (Low)

- The packaged app is not code-signed or notarised (no `osxSign` / `osxNotarize`), arm64 only, so other Macs will show Gatekeeper warnings.
- Shelling out to the `unzip` binary (`server.js:20, 24`) is a portability risk and an undeclared prerequisite; use a JS unzip library.
- `package.json` has no `engines` field although the README requires Node 20+, no lint or format scripts, no licence field.
- The delivery archive contains `.DS_Store`, `__MACOSX/`, about 120 MB of `out/` build output, the 88 MB `.cache/`, and a 4 MB `sncf-max-main.zip`.
- That last file is an unrelated third-party Python project (booking and scanning automation, no LICENSE file at its root). It is excluded from packaging and was not audited, but its provenance and licence should be confirmed before the project is shared.
- `appBundleId` embeds a personal name (`com.samwinter...`), which is cosmetic.

5. What the project does well

- Frontend output is consistently escaped (`esc()`), external links use `rel="noopener noreferrer"`, and error text is inserted with `textContent`.
- The server listens on `127.0.0.1` only and uses `execFile` with an argument array (no shell injection path); URLs are built with `URLSearchParams` / `encodeURIComponent`.
- No hard-coded credentials were found in the project's own source.

- The app has zero runtime dependencies, and degrades to cached data when the network fails.
- UI copy is candid about what the data can and cannot prove (snapshot flag, OUIGO unverified, Brussels eligibility). The audit scripts and sample mode are useful engineering aids.
- The existing tests target real past regressions (overnight connections, terminal changes, five-change Explore).

6. Suggested remediation order

1. **Safety net (hours):** top-level error handler, `Host` check, parameter validation with 400s (F-04 to F-07).
2. **Data freshness (a day):** timetable refresh with age notice; shared download promises; atomic swaps (F-01, F-08, F-09).
3. **Performance (days):** window-limited leg generation, early relevance filtering, worker thread (F-02, F-13).
4. **Correctness (days):** transfer-time floor and tests (F-03), boarding flags (F-12), explicit Paris timezone (F-11).
5. **Quality (ongoing):** fix the failing test, add server and rules tests, format and de-duplicate the code, add CI, upgrade Electron Forge and sign the build (F-19 to F-23).

7. Limitations

Live SNCF endpoints could not be reached from the audit environment, so the MAX download, GTFS download, ODSQL query behaviour and the Electron GUI were assessed by reading code, not by running them. Performance figures come from the shipped cached feed on a single run and will vary by machine; the relative scaling with date span is the meaningful result. Findings marked "By inspection" should be reproduced before being treated as confirmed.